



**UNIVERSITY OF NAIROBI
FACULTY OF SCIENCE AND TECHNOLOGY
DEPARTMENT OF COMPUTING AND INFORMATICS**

SentiNet: IoT Security Monitoring System

Hawatif Abdisalam

SCS3/148910/2024

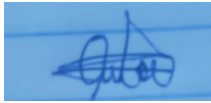
**A Project Report Submitted in Partial Fulfillment of the Requirements for the Award of
Bachelor of Science in Computer Science of the University of Nairobi**

June 2026

DECLARATION

This project is my original work and to the best of my knowledge this work has not been submitted for any other award in any University. All sources of information used in this report have been duly acknowledged.

Signed:



Date: 11-06-26

Hawatif Abdisalam

SCS3/148910/2024

This project report has been submitted in partial fulfillment of the requirements of the Bachelor of Science in Computer Science of the University of Nairobi with my approval as the University supervisor.

Date: 11-06-26

Christopher A. Moturi

Department of Computer Science

University of Nairobi

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Prof. Christopher A. Moturi, for his invaluable guidance, patience, and encouragement throughout the development of this project. His expertise and constructive feedback were instrumental in shaping the direction and quality of this work.

I am grateful to the Department of Computing and Informatics, University of Nairobi, for providing the academic environment and resources that supported this project.

I would like to thank my family and friends for their unwavering support and encouragement during my studies. Their belief in my abilities gave me the motivation to persevere through the challenges encountered during this project.

Finally, I acknowledge the open-source community behind tools such as Flask, Nmap, scikit-learn, and Docker, whose freely available tools made the technical implementation of SentiNet possible.

ABSTRACT

The rapid adoption of Internet of Things (IoT) devices in Small and Medium Enterprises (SMEs) in Kenya has increased security risks due to limited visibility and management of connected devices such as cameras, printers and sensors. Between April and June 2025, Kenya recorded over 4.5 billion cyber threat events, highlighting the need for affordable IoT security solutions.

The aim of this project was to develop a lightweight, web-based IoT security monitoring system that provides SMEs with automated device discovery, classification, vulnerability intelligence, and security policy enforcement through a centralised dashboard.

SentiNet was developed using the Agile Software Development Methodology. The backend was built using Python and Flask, the frontend using HTML5 and JavaScript, and data was stored in a MySQL database hosted on XAMPP. Nmap was used for network scanning, a Random Forest machine learning model was used for device classification, and vulnerability information was retrieved from the National Vulnerability Database (NVD) API. Eleven Docker-based devices representing cameras, printers, wireless access points, and computers were deployed to simulate an SME network.

SentiNet successfully discovered and classified all simulated devices, retrieved CVE identifiers and CVSS scores from the NVD, performed automated checks for default credentials, outdated firmware, and risky open ports, and presented all results through a responsive centralised web dashboard with scan history and PDF report export.

The results demonstrate that automated IoT security monitoring for SMEs can be achieved using low-cost open-source technologies. However, testing was limited to simulated devices and vulnerability matching used keyword-based searches that may not always be precise.

Future work should include testing with real hardware, expanding machine learning dataset using real world scan data and improving vulnerability matching through the use of identifiers.

TABLE OF CONTENTS

DECLARATION	2
ACKNOWLEDGEMENT	3
ABSTRACT	4
LIST OF TABLES	10
Table 1: Objectives and Deliverables	10
ABBREVIATIONS	11
CHAPTER 1: INTRODUCTION	12
1.1. Background	12
1.2. Problem Statement	12
1.3. Objectives	13
1.4. Scope	14
1.5. Significance	14
1.6. Justification	15
1.7. Constraints	15
CHAPTER 2: LITERATURE REVIEW	16
2.1. Role of ICT in IoT Security Management	16
2.2. Challenges in IoT Security	16
2.3. Review of Existing Systems	17
2.4. Technologies Used in Similar Systems	18
2.3. Gaps in the Current Systems	19
2.4. The Proposed System	20
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN	21
3.1. Systems Analysis and Design Methodology	21
3.2. System Analysis	22
3.2.1. Requirements Elicitation	22
3.2.2. Requirements Analysis	23
3.2.2.1. Functional Requirements	23
3.2.2.2. Non Functional Requirements	24
3.3. System Design	25
3.3.1. Architectural Design	25
3.3.2. Processes Design	25
3.3.3. Database Design	27
3.3.4. Application Design	28
3.3.5. User Interface Design	29

3.3.6. Integration Design	30
3.3.7. Technology Design	31
3.4. Resources	32
3.5. Schedule and Gantt Chart	33
CHAPTER 4: SYSTEM IMPLEMENTATION	33
4.1. Design and Implementation Platform	34
4.2. Installation and Setup	35
4.3. Testing	36
4.4. Maintenance Plan	37
4.5. Security and Ethical Considerations	38
CHAPTER 5: CONCLUSION AND RECOMMENDATION	39
5.1. Achievements	40
5.2. Limitations	41
5.3. Recommendations	41
5.4. Conclusion	42
REFERENCES	44
APPENDICES	46
Appendix A: SentiNet System User Guide	46
A.1 System Requirements	46
A.2 Downloading and Installing the System	46
A.2.1 Cloning the Repository from GitHub	46
A.2.2 Creating the Python Virtual Environment	47
A.2.3 Installing Python Dependencies	47
A.2.4 Starting the Flask Application	47
A.3 Accessing the System	47
A.4 Creating a New Account	48
A.5 Signing In	49
A.6 Dashboard Overview	50
A.7 Running a Network Scan	51
A.8 Device Cards and Security Findings	52
A.9 Filtering Devices	53
A.10 Reports and Analytics	53
A.11 Security Centre	54
A.12 SentiNet AI Assistant	55
A.13 Troubleshooting	58
Appendix B: Sample Code	60

LIST OF FIGURES

Figure 1: Agile Development Process
Figure 2: Architectural Design
Figure 3: Context Diagram
Figure 4: Level 1 DFD Diagram
Figure 5: Database Design
Figure 6: Application Design
Figure 7: Landing Page Design
Figure 8: Sign in Page Design
Figure 9: Integration Design
Figure 10: Technology Design
Figure 11: Gantt Chart
Figure A.1: Flask Backend Server Running Successfully
Figure A.2: SentiNet Landing Page
Figure A.3: Account Registration Form
Figure A.4: Sign In Page
Figure A.5: Main Dashboard Before Scanning
Figure A.6: Network Scan in Progress
Figure A.7: Dashboard Showing Scan Results with Device Cards
Figure A.8: Expanded Device Card Showing Security Findings and CVE Data
Figure A.9: Device List Filtered by IP Camera Category
Figure A.10: Reports and Analytics Page with Risk Distribution and Device Mix Charts
Figure A.11: Security Centre Showing Prioritized Recommendations
Figure A.12: SentiNet AI Assistant Responding to a Security Query
Figure A.13: Incidents View Listing Open Security Issues
Figure A.14: Security Awareness Module Showing Available IoT Security Topics

Figure B.1: Sample Code

LIST OF TABLES

Table 1: Objectives and Deliverables

Table 2: Schedule Table

Table 3: System Requirements

Table 4: Summary Statistics

Table 5: Troubleshooting

ABBREVIATIONS

API - Application Programming Interface

CCTV - Closed Circuit Television

CPS - Cyber-Physical Systems

DDoS - Distributed Denial-of-Service

DFD - Data Flow Diagram

ICT - Information and Communication Technology

IDS - Intrusion Detection System

IloT - Industrial Internet of Things

IoMT - Internet of Medical Things

IoT - Internet of Things

IP - Internet Protocol

IT - Information Technology

LAN - Local Area Network

NVD - National Vulnerability Database

OT - Operations Technology

SIEM - Security Information and Event Management

SME - Small and Medium Enterprise

VS Code - Visual Studio Code

WAP - Wireless Access Point

CHAPTER 1: INTRODUCTION

1.1. Background

The Internet of Things (IoT) refers to a vast network of interconnected physical devices that collect and exchange data over the internet. These “things” utilize embedded systems to communicate via Local Area Networks (LAN) and cloud platforms to provide real-time monitoring and automation. While IoT has revolutionized operational efficiency, conventional security solutions prove inadequate due to the wide variety of devices, massive data generated and a significant visibility gap, where organizations lack full insight into all connected devices and their activity, creating blind spots that attackers can exploit (Adewuyi et al., 2024).

In Kenya, the adoption of IoT technologies has been steadily increasing. Machine-to-Machine subscriptions, a key measure of IoT deployment, have risen by 3.5% to nearly two million, reflecting the growing integration of these devices across industries (Communications Authority of Kenya, 2025). This growth is particularly evident in urban centers like Nairobi, where Small and Medium Enterprises (SMEs) are increasingly deploying “smart office” solutions (Ouma et al., 2026),

Despite this rapid adoption, the security of these devices remains a concern. Between the period of April-June 2025, the country experienced 4.5 billion cyber threat events, an increase of more than 80% compared to January-March 2025. These prevailing global trends can be largely attributed to the continued proliferation of Internet of Things (IoT) devices, many of which are deployed without adequate security controls (National KE-CIRT/CC, 2025).

The purpose of the proposed system is to support SMEs and other users in managing the security of their IoT devices by providing better visibility and monitoring, helping to detect cyber threats in a timely manner.

1.2. Problem Statement

Small and Medium Enterprises (SMEs) and small office environments in Kenya increasingly rely on IoT devices such as network cameras, printers, sensors and smart plugs to support daily operations. However, these devices are deployed within LAN environments without adequate security monitoring or visibility. As a result, the users face heightened risk of cyber incidents, including unauthorized access, device compromise, and network misuse, which may go

undetected due to the lack of continuous monitoring mechanisms. If this problem is not addressed, insecure IoT deployments may continue to serve as entry points for cyber attacks, contributing to data breaches, service disruptions, and participation in large-scale threats such as Distributed Denial-of-Service (DDoS) attacks. This challenge is increasingly evident in urban small office and SME environments, where the deployment of IoT devices has grown faster than the implementation of corresponding security monitoring mechanisms. Addressing this issue can improve network visibility and reduce exposure to potential threats.

1.3. Objectives

1.3.1. Main Objective

To develop a web based security management and policy enforcement system that enhances visibility, monitoring and protection of IoT devices within SME network environments.

1.3.2. Specific Objectives

1. To identify gaps in the existing IoT deployment and security management practices in SMEs.
2. To design and develop a web based IoT security monitoring system.
3. To integrate security monitoring mechanisms to a central unified platform.

How the objective has been achieved

Table 1: Objectives and deliverables

Objective	How it will be achieved	Deliverable
To identify gaps in the existing IoT deployment and security management practices in SMEs.	Through literature review and analysis of current IoT deployment practices within SME environments to determine weaknesses in device visibility, monitoring, and policy enforcement.	Clearly defined system requirements and identified security gaps that guide the system design.

To design and develop a web based IoT security monitoring system,	By designing system architecture, developing a frontend dashboard, implementing backend scanning and monitoring modules, and integrating a database for storing device and security data.	A functional web-based IoT security monitoring prototype capable of device discovery, port analysis, firmware checking, and policy enforcement.
To integrate security monitoring mechanisms to a central unified platform.	By combining network scanning, device identification, firmware validation, password checks, behavioural monitoring, and vulnerability alert generation within one coordinated system interface.	A centralized dashboard that provides real-time device visibility, security status indicators, and automated alert notifications.

1.4. Scope

The proposed project primarily focuses on enhancing the security and management of IoT devices in SMEs and home office environments. It targets key users such as office managers and IT administrators, who need better visibility and control over these devices.

1.5. Significance

The proposed IoT security monitoring system is significant to SMEs and their stakeholders by strengthening cybersecurity management in increasingly connected work environments. For SME owners and managers, the system provides visibility into all connected devices, enabling them to reduce financial and operational risks associated with cyber incidents. For IT administrators or technical personnel, it offers a centralized platform for monitoring device security status, enforcing policies, and responding to vulnerabilities in a timely manner. Employees benefit from improved system reliability and reduced service disruptions caused by compromised devices, while customers gain greater assurance that their data and transactions are protected. Additionally, the system supports regulators and policymakers by encouraging improved cybersecurity compliance and responsible IoT deployment within small and medium

enterprises. Overall, the project contributes to improved digital trust, operational continuity, and sustainable technological growth within Kenya’s SME sector.

1.6. Justification

This project aligns with the University of Nairobi’s “Big 5” initiatives, which focus on innovation, digital transformation, and societal impact. By improving the security and management of IoT devices, the proposed system directly supports the UoN Innovation Park and the Kenya Green Jobs Center, initiatives that encourage technological innovation, skills development, and the creation of safer, more resilient digital ecosystems (University of Nairobi, 2024).

1.7. Constraints

Limited access to real IoT devices, hence the system will rely on the simulation of these devices for testing and validation.

Ethical and legal considerations restrict network scanning to authorized environments only.

Variations in device manufactures, firmware structures and communication protocols may affect accurate identification.

CHAPTER 2: REVIEW OF SIMILAR SYSTEMS

2.1. Role of ICT in IoT Security Management

Information and Communication Technology (ICT) plays a critical role in securing Internet of Things (IoT) environments in Kenya, especially in urban centers like Nairobi where smart devices are increasingly used in homes, offices, banks, hospitals, and learning institutions. IoT security management involves using ICT tools such as network monitoring systems, vulnerability scanners, intrusion detection systems (IDS), encryption technologies, cloud-based dashboards, and security information and event management (SIEM) systems to monitor, detect, and respond to threats affecting connected devices. For instance, a smart security framework utilizing PIR sensors and microcontrollers can automate this detection and response process by capturing images of intruders and alerting owners in real-time (Mahendra et al., 2018).

In Kenya, national ICT and cybersecurity governance is guided by institutions such as the Communications Authority of Kenya and the Kenya Computer Incident Response Team Coordination Centre (KE-CIRT/CC), which coordinate incident response and cybersecurity awareness. The legal framework is supported by the Computer Misuse and Cybercrimes Act, which criminalizes unauthorized access, cyber espionage, and other digital offenses. At the organizational level, the Chief Information Officer (CIO) plays a pivotal role in this transformation, as their innovative capacity and technical expertise are directly linked to the successful adoption and securing of these emerging technologies (Parra & Guerrero, 2020).

2.2. Challenges in IoT Security

Despite progress in ICT adoption, IoT security management in Kenya faces several challenges. One major issue is the widespread use of low-cost IoT devices that often lack built-in security features or regular firmware updates. Many small businesses and households in Nairobi deploy routers, CCTV cameras, and smart devices without changing default passwords, making them easy targets for cybercriminals. This vulnerability is compounded by the fact that many organizations do not prioritize security during the initial planning stages of IoT projects, often ignoring best security practices in favor of rapid deployment (Kuzminykh et al., 2020).

Another challenge is limited cybersecurity awareness and technical capacity. While institutions like KE-CIRT/CC provide guidance, many organizations lack dedicated security teams or structured IoT security policies. Furthermore, the decision to adopt IoT is often hindered by high investment costs related to technology acquisition, maintenance, and the specialized human resources required to manage such complex ecosystems (Suciu et al., 2021). Additionally, variations in device manufacturers, firmware architectures, and communication protocols make standardization and monitoring difficult. Cloud hosting also introduces concerns about data sovereignty, privacy, and compliance with local regulations.

Finally, financial constraints, limited enforcement of cybersecurity policies, and the fast growth of connected devices outpacing regulatory controls contribute to gaps in IoT security implementation across Kenya.

2.3. Review of Existing Systems

2.3.1 Armis Centrix

Armis Centrix is an agentless security platform that provides real-time monitoring and asset management for IT, OT, and IoT devices. It enables organizations to identify, track, and secure connected devices, providing visibility into device behavior, risk assessment, and policy enforcement (Armis Centrix, n.d.).

2.3.2 Forescout 4D Platform

Forescout 4D Platform delivers automated security and compliance for connected devices without requiring agents. It continuously discovers and classifies devices on the network, enforces policies, and integrates with existing IT and security systems to maintain organizational compliance (Forescout, n.d.).

2.3.3 Palo Alto Networks IoT Security

Palo Alto Networks IoT Security specializes in securing IoT environments by providing device visibility, risk assessment, and policy enforcement. It is particularly suited for industrial and healthcare networks, monitoring devices for vulnerabilities and suspicious behavior (Palo Alto Networks, n.d.).

2.3.4 Claroty Platform

Claroty focuses on cyber-physical systems (CPS) protection in industrial (IIoT) and medical (IoMT) networks. It provides asset management, secure access, and vulnerability monitoring to safeguard operational technology from cyber threats (Claroty, n.d.).

2.3.5 Microsoft Defender for IoT

Microsoft Defender for IoT is an agentless security solution that provides device discovery, behavioral analytics, and threat intelligence. It supports diverse IoT devices and networks, offering risk assessment and continuous monitoring (Microsoft Defender, n.d.).

2.3.6 Fanan Limited

Fanan Limited offers mobile and IoT endpoint security, focusing on real-time threat detection, prevention, and response. The platform aims to secure connected devices by monitoring activity and enforcing basic security policies (Fanan Limited, n.d.).

2.3.7 Opticom Kenya Limited

Opticom Kenya provides IoT-powered security solutions, integrating surveillance and safety technologies for homes, offices, and small businesses. Their focus is on IoT device monitoring, safety automation, and policy enforcement (Opticom Kenya Limited, n.d.).

2.4. Technologies Used in Similar Systems

Agentless Network Discovery

Platforms such as Armis Centrix, Forescout 4D, and Microsoft Defender for IoT use agentless scanning to detect devices connected to a network. This allows administrators to identify and classify IoT and IT devices without installing software agents on each device.

Device Classification and Profiling

Tools like Forescout and Armis classify devices automatically (printers, cameras, WAPs, industrial controllers) based on traffic, behavior, and fingerprinting. Classification enables policy enforcement and targeted monitoring.

Risk Assessment and Vulnerability Analysis

Systems such as Palo Alto Networks, Claroty, and Microsoft Defender for IoT assess device risks, detect vulnerabilities, and flag suspicious behavior. They often integrate with vulnerability databases (CVE/NVD) or proprietary threat intelligence feeds.

Policy Enforcement

Enterprise platforms and Kenyan providers (Fanan, Opticom) allow for policy application, such as blocking unauthorized access, enforcing passwords, or limiting device communication. Policies can vary by device type and intended use.

Behavioral Analytics

Solutions like Microsoft Defender for IoT and Fanan Limited use behavioral monitoring to identify anomalies, unusual traffic patterns, or potential threats.

Surveillance & IoT Integration (Kenya-Specific)

Opticom Kenya Limited integrates IoT devices with surveillance and automation for homes and offices, providing safety monitoring alongside basic IoT security.

Endpoint Security and Threat Detection

Fanan Limited focuses on endpoint-level protection, including real-time threat detection, prevention, and response for mobile and IoT devices.

2.3. Gaps in the Current Systems

1. Most global platforms, including Armis, Forescout, Palo Alto Networks, and Claroty, are designed for enterprise, industrial, or healthcare environments, while local providers such as Fanan and Opticom may target small deployments but often through service-based implementations rather than automated solutions for SMEs.
2. Kenyan providers mainly focus on audits, installation, or manual monitoring, and automated scanning, classification, and real-time enforcement for everyday office devices such as printers, cameras, and WAPs is rare or absent.
3. Most systems assess risk at a single point in time or rely heavily on cloud-based intelligence. SMEs in Kenya require localized, lightweight solutions that check firmware, open ports, running services, and NVD vulnerabilities continuously.
4. Enterprise tools often demand trained cybersecurity personnel to configure, interpret alerts, and enforce policies, which is impractical for SMEs lacking in-house expertise.
5. Global solutions also rely on cloud dashboards, which may be unreliable or costly for Kenyan SMEs with limited bandwidth, and local providers may require service visits rather than offering continuous monitoring through a user-friendly interface.

2.4. The Proposed System

The proposed IoT Device Security Monitoring and Policy Enforcement System is designed for SMEs in Kenya. It automatically scans and discovers devices on the network, classifies them as IoT or non-IoT, and groups IoT devices into categories such as printers, cameras, and WAPs. For each category, it applies security policies, checks for default credentials, outdated firmware, open ports, and running services, and integrates with the National Vulnerability Database (NVD) to provide real-time alerts on vulnerabilities.

The system features a centralized, web-based dashboard that presents all monitoring and alert information in a simple interface. It is lightweight and cost-effective. By combining automated discovery, policy enforcement, real-time vulnerability alerts, and human-centered monitoring, the system provides SMEs with a practical, accessible, and effective solution for securing their IoT devices and networks.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1. Systems Analysis and Design Methodology

The Agile Software Development Methodology was adopted for this project due to its flexibility and iterative nature. Agile supports incremental system development, allowing features to be built, tested, and improved continuously throughout the development lifecycle. This approach enables early development of a working prototype, particularly the frontend dashboard, followed by backend integration and device simulation in subsequent iterations.

Agile promotes continuous feedback and refinement, ensuring that system functionalities such as device monitoring, policy enforcement, alert generation, and dashboard visualization are progressively improved. The methodology also allows adjustments when introducing new device types (e.g., printers, IP cameras, sensors) or modifying compliance rules. These advantages make Agile suitable for developing a scalable and adaptable IoT monitoring system.

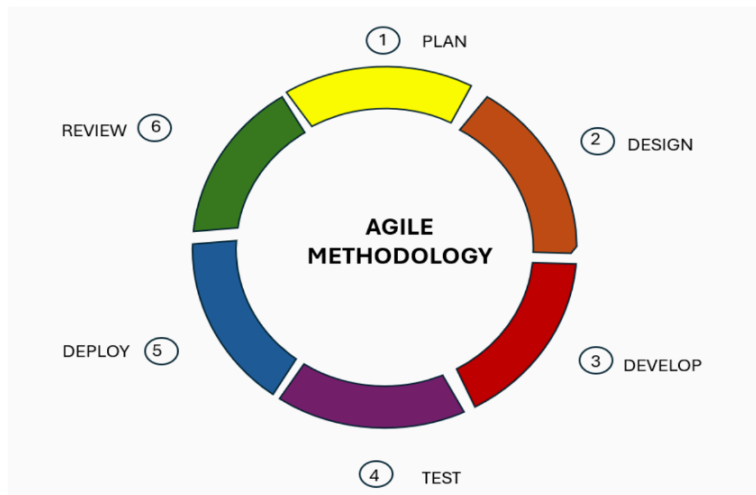


Figure 1: Agile Development Process

3.2. System Analysis

3.2.1. Requirements Elicitation

To gather system requirements, a combination of stakeholder consultation, system research, and use case modeling will be utilized to ensure thorough understanding of both the functional and non functional requirements.

Interviews: Interviews will be conducted with IT administrators, network managers, and system operators to identify the current challenges in monitoring IoT devices within organizational networks.

Questionnaires: Structured questionnaires will be distributed to potential users to gather detailed insights regarding commonly deployed IoT devices (such as IP cameras, printers, sensors, and smart plugs), common security concerns, and desired system features.

Use Cases: Use cases will be developed to illustrate how different users will interact with the system. The primary users will include administrators and IT operators.

Through this elicitation process, clear system objectives and constraints will be identified, ensuring that the final system design aligns with user needs and organizational security requirements.

Developing the IoT Device Monitoring and Policy Compliance System will lead to significant improvements in efficiency and community participation:

1. **Centralized Device Visibility:** The system will provide a unified dashboard displaying all monitored devices, their status, and compliance level.
2. **Automated Policy Enforcement:** Security policies such as password strength requirements, firmware update checks, and port restrictions will be automatically evaluated.
3. **Real-Time Alerting:** The system will generate alerts when devices violate defined policies or show abnormal behavior.
4. **Improved Security Awareness:** By visualizing compliance statistics and risk levels, administrators can quickly identify vulnerable devices and take corrective action.

3.2.2. Requirements Analysis

Based on the elicited requirements, the system will provide the following functionalities:

3.2.2.1. Functional Requirements

User Management:

The system shall allow administrators to log in securely.

Device Monitoring:

The system shall display a list of all simulated IoT devices.

The system shall categorize devices by type (IP camera, printer, sensor, smart plug).

The system shall display device details such as IP address, manufacturer, firmware version, open ports, and status (online/offline).

Device Simulation:

The system shall simulate IoT devices using predefined datasets or scripts.

The system shall generate simulated device activity and policy violations for demonstration purposes.

Policy Management:

The system shall define security policies (e.g no default passwords, firmware must be up to date, restricted open ports).

The system shall evaluate each device against defined policies.

Alert Generation:

The system shall generate alerts when a device fails a compliance check.

Alerts shall be displayed on the dashboard with severity levels (Low, Medium, High).

Reporting and Dashboard:

The system shall provide visual statistics (charts and graphs) showing compliance rates and device distribution.

The system shall allow filtering devices by type, manufacturer, and compliance status.

3.2.2.2. Non Functional Requirements

Performance:

The system shall process simulated device data and generate compliance results within 60 seconds.

The system shall support multiple concurrent users without performance degradation.

Usability:

The dashboard interface shall be simple, responsive, and easy to navigate.

The system shall provide clear labels, icons, and visual indicators for device status and alerts.

Accessibility:

The system shall be accessible via modern web browsers.

The interface shall be responsive for desktop and mobile devices.

Security:

User authentication credentials shall be securely stored using encryption or hashing.

Access to administrative functions shall be restricted to authorized users.

Reliability:

The system shall maintain at least 99% uptime during operation.

Backup mechanisms shall be implemented to prevent data loss.

3.3. System Design

3.3.1. Architectural Design

Figure 2 shows the architecture design of the system showing its structural components.

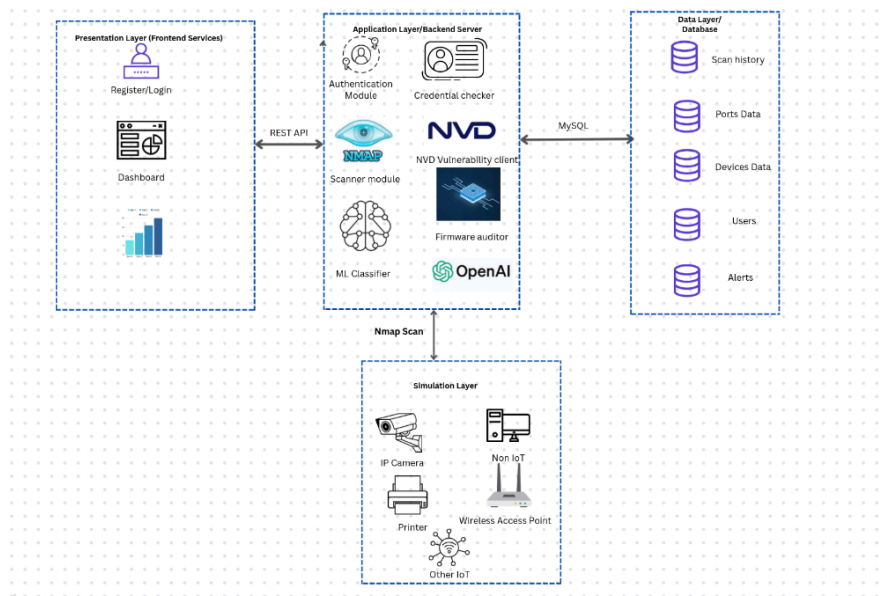


Figure 2: Architectural Design

3.3.2. Processes Design

Figures 3 and 4 illustrate the workflows and interactions within the System, providing a clear overview of the system's operational processes

Context Diagram

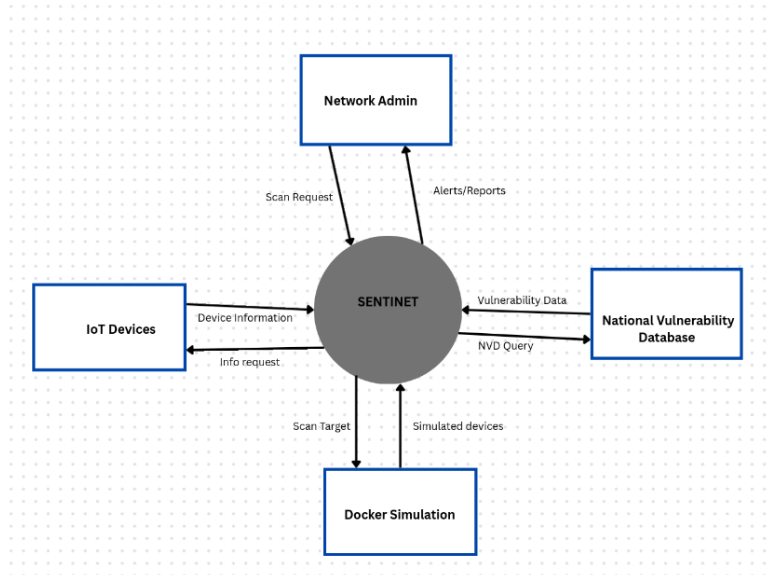


Figure 3: Context Diagram

DFD Diagram

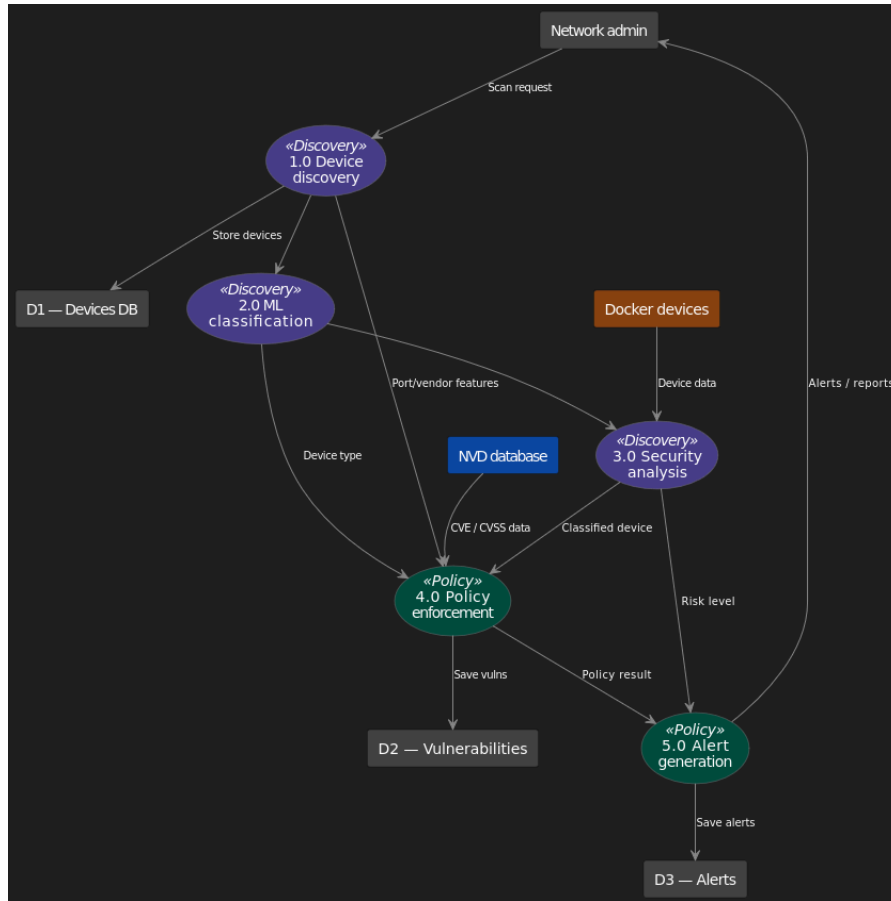


Figure 4: Level 1 DFD Diagram

3.3.3. Database Design

Database Design Diagram in figure 5 illustrates the structure and organization of the IoT Security Management systems database. This diagram provides a visual representation of the tables, relationships, and key attributes within the database

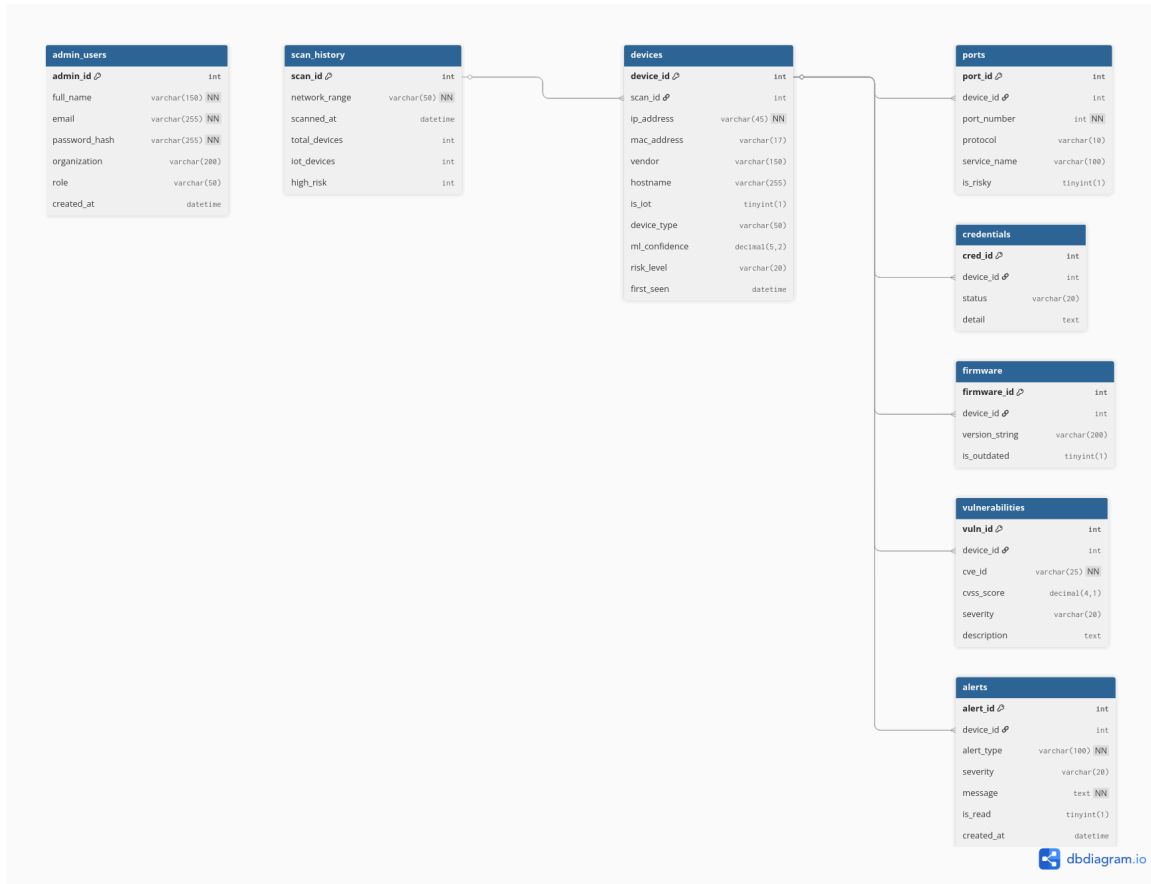


Figure 5: Database Design

3.3.4. Application Design

The applications for the system's user are illustrated in Figure 6 below.

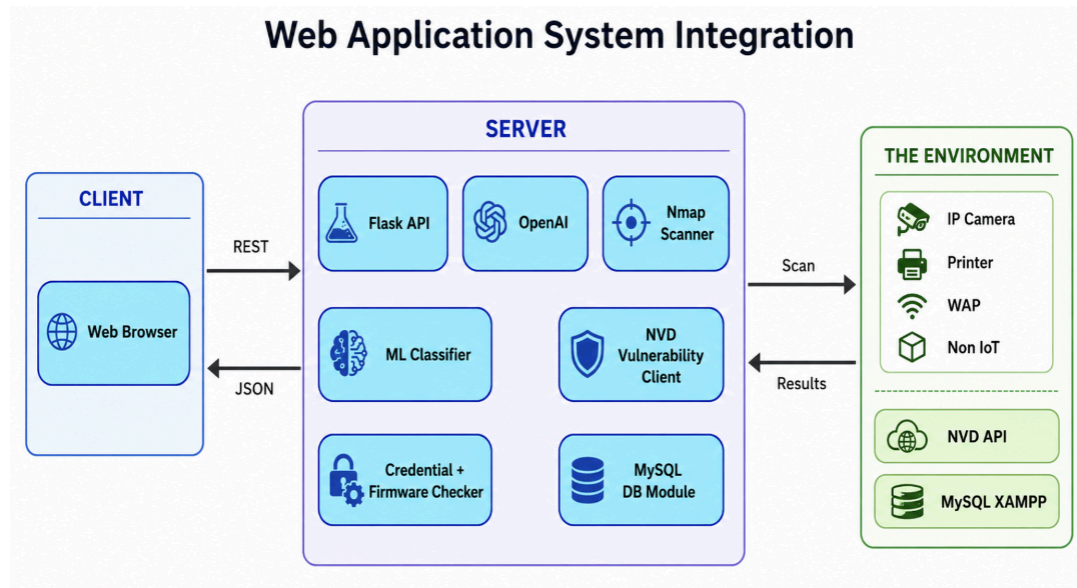


Figure 6: Application Design

3.3.5. User Interface Design

Figure 7 and 8 shows the user interface anticipating user expectations for effective access.

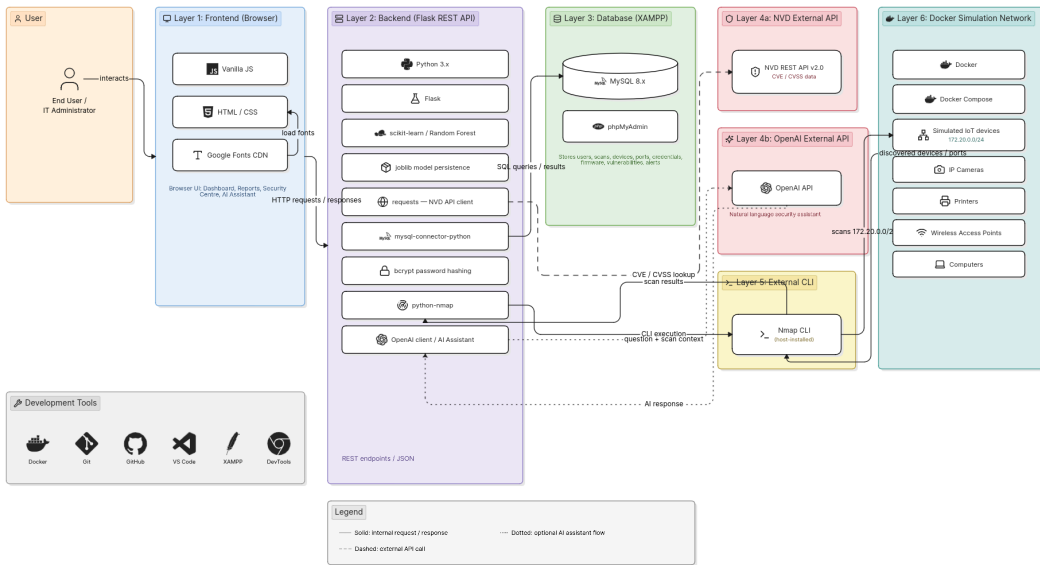


Figure 7: Landing page

3.3.6. Integration Design

Figure 8 illustrates how various components will work together including Services, Processes, Events and Data

Web Application System Integration — SentiNet IoT Security Monitoring System



eraser

Figure 9: Integration Design

3.3.7. Technology Design

Figure 9 defines how technologies in terms of Infrastructure, Systems, Applications, Toolset, Libraries, APIs

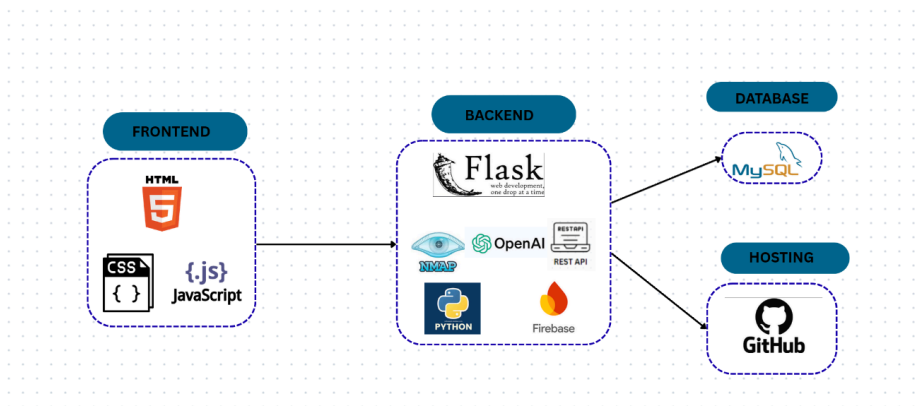


Figure 10: Technology Design

3.4. Resources

1. Hardware

Development Machine: A laptop or desktop with at least 8GB RAM, Intel i5 processor (or higher), and 256GB SSD storage for smooth coding, testing, and running development environments.

Testing Devices: A variety of smartphones, tablets, and desktops to ensure the system is responsive across different screen sizes.

Cloud Hosting Server: A reliable cloud service (AWS, DigitalOcean, or Firebase) for deploying the backend and database securely.

2. Software & Development Tools

Integrated Development Environment (IDE): Visual Studio Code for writing and debugging code.

Database Management Tool: MySQL for designing and managing database structures.

Version Control & Collaboration: Git & GitHub for source code management and team collaboration.

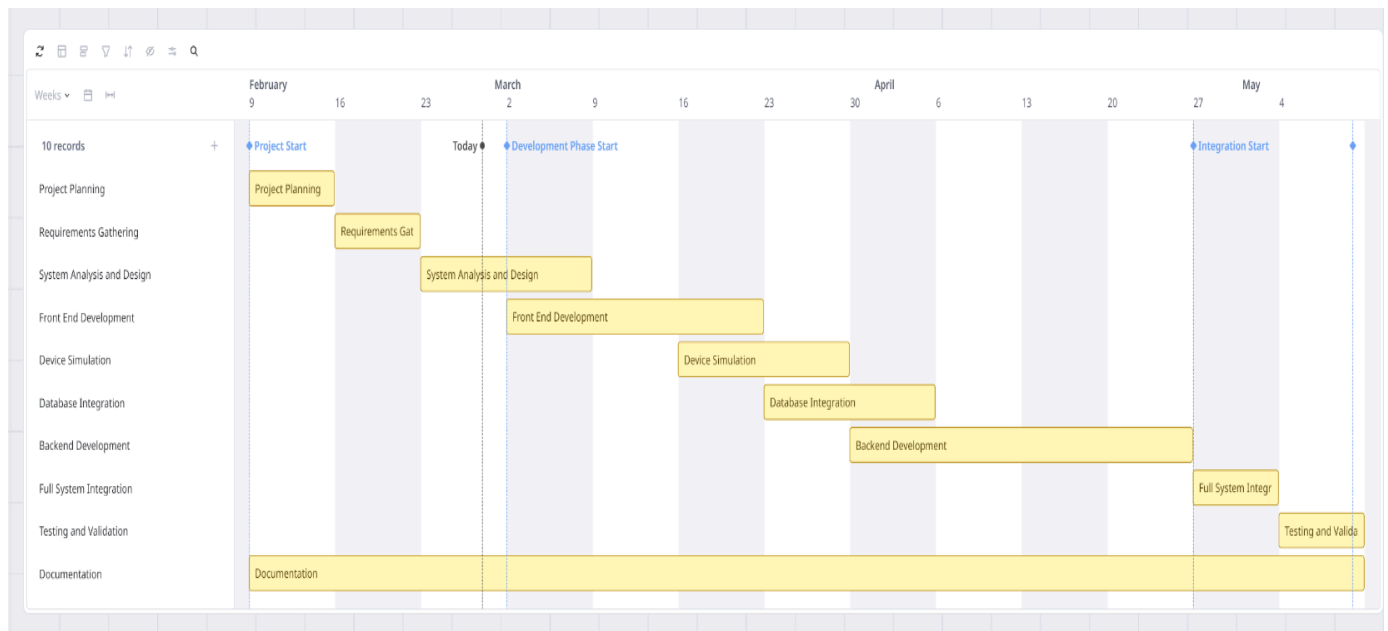
API Testing Tool: Postman for testing REST API requests and responses.

3.5. Schedule and Gantt Chart

Table 2: Schedule table

Task No	Task Name	Hours	Planned Start Date
1	Project Planning	15	09/02/26
2	Requirements Gathering	20	16/02/26
3	System Analysis and Design	20	23/02/26
4	Front End Development	45	02/03/26
5	Device Simulation	35	16/03/26
6	Database Integration	20	23/03/26
7	Backend Development	55	30/03/26

8	Full System Integration	30	27/04/26
9	Testing and Validation	25	04/05/26
10	Documentation	35	09/02/25



CHAPTER 4: SYSTEM IMPLEMENTATION

4.1. Design and Implementation Platform

SentiNet was developed as a web-based IoT security monitoring system designed to help Small and Medium Enterprises (SMEs) in Kenya gain visibility over connected devices on their networks, detect security threats, and enforce device security policies. The system integrates network scanning, machine learning classification, real-time vulnerability intelligence, and a centralized web dashboard into a unified platform.

The frontend of the platform was built using HTML5, CSS3, and vanilla JavaScript, creating a responsive and intuitive user interface. The design follows a white, purple, and blue colour scheme to reflect a professional and technology-oriented aesthetic. The interface includes a landing page, a sign in and account registration page, a network scanning dashboard, and a reports and analytics page, all styled for clarity and ease of use.

The backend was implemented entirely in Python using the Flask framework, which exposes a RESTful API consumed by the frontend. The backend orchestrates the core intelligence of the system, encompassing network scanning via Nmap, feature extraction from scan results, device classification using a trained Random Forest machine learning model, firmware version auditing, default credential detection, vulnerability lookups against the National Vulnerability Database (NVD), policy compliance evaluation, and risk scoring.

A MySQL relational database hosted on XAMPP was used to store and manage all system data. The database is structured around eight core tables: scan_history, devices, ports, credentials, firmware, vulnerabilities, alerts, and admin_users. These tables collectively store scan records, device profiles, open port details, credential check outcomes, firmware versions, NVD-sourced CVE entries, generated security alerts, and administrator accounts. The schema was designed to enforce referential integrity through foreign key relationships and to support efficient querying by the Flask API layer.

Device simulation was implemented using Docker, with eleven containerised devices deployed on a dedicated subnet (172.20.0.0/24). The simulated environment includes four IP cameras from manufacturers such as Hikvision and Dahua, three network printers from HP, Canon, and Epson,

two wireless access points from TP-Link and Cisco, and two computers representing Windows workstations. Each container exposes the ports and HTTP responses characteristic of its device type, enabling the Nmap scanner to discover and fingerprint them as real devices would be treated on a live SME network.

4.3. Testing

A comprehensive testing strategy was implemented to verify that SentiNet met its functional and non-functional requirements. The testing process covered unit testing, integration testing, pipeline testing, and security validation.

Unit testing was conducted on individual Python modules to confirm that each component performed its function correctly in isolation. The feature extraction module was tested to ensure that device scan dictionaries were consistently converted into fixed-length numerical feature vectors using the defined `FEATURE_ORDER` list. The rule-based labeller in the dataset builder was tested with simulated device profiles to verify that classification priorities were correctly applied, ensuring that cameras, printers, computers, and WAPs were labelled without ambiguity. The credential checker and firmware auditor were tested against sample HTTP responses to confirm accurate extraction of device metadata.

Integration testing evaluated the end-to-end pipeline from network scanning through to database storage. A dedicated test script, `test_pipeline.py`, was developed to orchestrate the full workflow: training the machine learning model, scanning the Docker simulation subnet, classifying all discovered devices, saving results to MySQL, and printing a structured scan report. This test confirmed that all system components communicated correctly and that data flowed without loss or distortion from the Nmap scanner through to the database and dashboard.

The machine learning classifier was evaluated using a train-test split on the combined synthetic and real scan dataset. Classification accuracy and a per-class report were generated to assess the model's performance across all device categories. A confidence threshold of 60 percent was enforced to ensure that low-confidence predictions were returned as unknown rather than propagated as incorrect labels. The top five most influential features identified by the Random

Forest model were also recorded to support explainability and provide evidence of meaningful learning.

NVD integration was tested by submitting known vendor and service keyword queries to the NVD REST API v2.0 and verifying that CVE identifiers, CVSS scores, and severity classifications were correctly parsed and stored against the corresponding device records. The risk scoring engine was tested with combinations of CVSS score, open port flags, and credential status to confirm that risk levels of low, medium, high, and critical were assigned as expected.

Security validation confirmed that administrator passwords were stored as bcrypt hashes and never returned in API responses. Authentication flows were verified to reject incorrect credentials and to update the last login timestamp on successful sign-in. Input fields on the registration form were tested for completeness validation, and the terms of agreement checkbox was confirmed to gate account creation correctly.

4.4. Maintenance Plan

The maintenance plan for SentiNet has been structured to support long-term operational stability, ensure data integrity, and keep the system aligned with evolving security intelligence.

Regular backups of the MySQL database are scheduled to prevent data loss and enable recovery in the event of system failures. Backups cover all tables including device records, vulnerability data, scan history, and alert logs.

The NVD integration relies on the NVD REST API v2.0, which is maintained by the National Institute of Standards and Technology. Periodic rescans of stored devices are recommended to refresh vulnerability data as new CVEs are published. The system architecture supports re-querying the NVD for any stored device without requiring a full network rescan.

The machine learning model is expected to improve over time as more real scan data is collected. A retraining workflow is built into the pipeline: any new scan results that are manually labelled can be fed into the training dataset, and classifier.py can be re-run to produce an updated model file. The modular architecture ensures that the updated model is immediately available to the prediction pipeline without changes to other system components.

4.5. Security and Ethical Considerations

To maintain user trust and ensure the protection of sensitive network data, SentiNet implements a set of security and ethical safeguards aligned with the system's acceptable use terms.

All administrator passwords are hashed using bcrypt before storage and are never transmitted or returned in plain text by any API endpoint. Access to administrative functions is restricted to authenticated users, and session management is handled to prevent unauthorized access to the dashboard.

Network scanning is ethically constrained by the system's terms of agreement, which users must explicitly accept during account registration. The terms require users to confirm that they will only scan IP addresses and networks belonging to their registered organisation or for which they have explicit written authorisation. This aligns with the legal framework established by the Computer Misuse and Cybercrimes Act in Kenya, which criminalises unauthorised access to computer systems.

Vulnerability data retrieved from the NVD is used solely for informational and defensive purposes within the platform. CVE identifiers and CVSS scores are displayed to administrators to support informed remediation decisions and are not shared externally or used to facilitate offensive security activities.

Device scan results, network topology data, and vulnerability reports stored in the database are treated as confidential organisational information. The system does not transmit this data to external services beyond the NVD API query, which uses only product name and service version keywords and does not expose internal IP addresses or network structure.

The use of Docker for device simulation, rather than scanning real third-party infrastructure, ensures that all testing activities during development remain within a controlled and authorised environment. This approach reflects the constraint identified in the project proposal regarding limited access to real IoT devices and demonstrates responsible handling of that limitation.

CHAPTER 5: CONCLUSION AND RECOMMENDATION

5.1. Summary Achievements

The aim of the project was to develop SentiNet, a web-based IoT security monitoring system that enhances visibility, monitoring, and protection of IoT devices within SME network environments. The system successfully achieved several key milestones:

Automated IoT Device Discovery and Classification: SentiNet successfully integrates Nmap network scanning with a trained Random Forest machine learning classifier to automatically discover devices on a network, extract meaningful features from scan results, and classify each device as a camera, printer, wireless access point, computer, or unknown. The classifier demonstrated strong performance on the training dataset and consistently identified device types across the Docker simulation environment.

Real-World Vulnerability Intelligence via NVD: The system successfully integrates with the National Vulnerability Database REST API v2.0 to retrieve CVE identifiers and CVSS scores for discovered devices based on vendor and service information. This gives SME administrators access to real-world vulnerability data directly within their dashboard, replacing vague risk descriptions with specific, actionable CVE records.

Automated Security Checks: SentiNet performs automated checks for default credentials, outdated firmware versions, and risky open ports for each discovered device. These checks are applied per device category using a policy enforcement engine, and the results feed directly into a composite risk scoring system that assigns each device a risk level of low, medium, high, or critical.

Centralized Web Dashboard: A clean and responsive web dashboard was developed using HTML, CSS, and JavaScript, providing administrators with a unified view of all scanned devices, their types, risk levels, ML confidence scores, open ports, and associated CVEs. The dashboard supports real-time filtering by device type and displays scan history pulled from the MySQL database.

Realistic Device Simulation with Docker: Eleven simulated IoT and non-IoT devices were containerised using Docker and deployed on a dedicated subnet, providing a realistic and

controlled test environment for the scanner and classifier. The simulation includes cameras, printers, wireless access points, and computers, each exposing authentic port signatures and HTTP responses, enabling end-to-end system validation without requiring access to real IoT hardware.

Secure User Authentication: Administrator accounts are managed securely with bcrypt password hashing and role-based access control.

5.2. Limitations

Dependence on Device Simulation: Due to limited access to real IoT devices, the system was validated primarily against Docker-simulated containers. While the simulation accurately replicates port signatures and HTTP responses, real-world devices may expose additional complexity, non-standard banners, or varied firmware structures that could affect classification accuracy and firmware detection reliability.

Heuristic Firmware Detection: Exact firmware version retrieval is not consistently achievable through Nmap alone. The system relies on banner grabbing and HTTP header parsing to infer firmware information, which may produce incomplete or missing results for devices that do not expose version details in their network responses.

Small Training Dataset: The machine learning model was trained primarily on synthetic data generated from predefined device profiles. While sufficient for demonstrating the classification pipeline, a larger and more diverse dataset drawn from real network scans would be needed to improve model generalisability across a wider range of device manufacturers and configurations.

5.3. Recommendations

Based on the development and testing of the SentiNet platform, the following recommendations are proposed for future enhancement:

Expand Real Device Testing: Conduct validation using a diverse range of real IoT devices in a controlled lab environment to identify gaps in classification accuracy, firmware detection, and credential checking that are not exposed by simulated containers. Real device testing will also enable the collection of labelled scan data to retrain and improve the machine learning model.

Implement Scheduled and Continuous Scanning: Introduce a background scanning scheduler that automatically rescans the network at configurable intervals and compares results against previous scans to detect newly connected devices, configuration changes, or newly published CVEs affecting existing devices.

Expand the Machine Learning Dataset: Grow the training dataset by incorporating scan results from real devices and a broader range of vendors. Techniques such as cross-validation and stratified sampling should be applied to improve model robustness. Additional device categories such as smart plugs, sensors, and industrial controllers could also be introduced.

5.4. Conclusion

SentiNet was developed as a prototype web-based IoT security monitoring system aimed at addressing the visibility gap that SMEs in Kenya face when managing connected devices on their networks. The project demonstrates that a functional and lightweight monitoring pipeline can be constructed using open-source tools, combining Nmap network scanning, rule-based and machine learning classification, NVD vulnerability lookups, and a web dashboard within a single integrated system.

The system was tested against a simulated environment of eleven Docker containers representing cameras, printers, wireless access points, and computers. Within this controlled setting, the pipeline successfully discovered devices, classified them by type, performed basic security checks, retrieved relevant CVE data, and stored results in a structured MySQL database for display on the dashboard.

While the prototype achieves its core objectives, it has notable limitations. Firmware detection relies on banner parsing and is not always reliable. NVD matching is keyword-based and may not always return precise results. Despite these constraints, the project provides a working proof of concept that demonstrates the feasibility of the proposed approach and lays groundwork for further development. With additional testing on real devices, a larger training dataset, and continuous scanning capabilities, the system could be extended into a more complete solution suited for practical SME deployment.

REFERENCES

1. Adewuyi, A., Oladele, A. A., Enyiorji, P. U., Ajayi, O. O., Tsambatare, T. E., Oloke, K., & Abijo, I. (2024). The convergence of cybersecurity, Internet of Things (IoT), and data analytics: Safeguarding smart ecosystems. *convergence*, 20, 21.
2. Armis Centrix, n.d. Armis Centrix: Agentless security platform for IT, OT, and IoT devices. [online] Available at: <https://www.armis.com/platform/armis-centrix/> [Accessed 25 February 2026].
3. Claroty, n.d. Claroty platform: Protecting industrial and medical cyber-physical systems. [online] Available at: <https://www.claroty.com/platform> [Accessed 25 February 2026].
4. Fanan Limited, n.d. Fanan Limited: Mobile and IoT endpoint security. [online] Available at: <https://fanansolutions.com/?srsltid=AfmBOoq3e4xqBxwe58SvCtnUYHMXhcidXFG1Xt2qMpouY4Iqc7mKwsh-> [Accessed 25 February 2026].
5. Forescout, n.d. Forescout 4D platform: Automated security and compliance for connected devices. [online] Available at: <https://www.forescout.com/> [Accessed 25 February 2026].
6. Kuzminykh, I., Ghita, B., & Such, J. M. (2020). The challenges with internet of things for business. *arXiv*.
7. Mahendra, S., Sathiyarayanan, M., & Vasu, R. B. (2018, August). Smart security system for businesses using internet of things (iot). In *2018 Second international conference on green computing and internet of things (ICGCIoT)* (pp. 424-429). IEEE.
8. Microsoft, n.d. Microsoft Defender for IoT: Agentless security solution for IoT devices. [online] Available at: <https://www.microsoft.com/en-us/security/business/endpoint-security/microsoft-defender-iot> [Accessed 25 February 2026].
9. National KE-CIRT/CC. (2025). Cyber threat report: April–June 2025. National KE-CIRT/CC, 1–15.
10. Opticom Kenya Limited, n.d. Opticom Kenya: IoT-powered security solutions. [online] Available at: <https://www.opticom.co.ke> [Accessed 1 March 2026].
11. Ouma, B. O., Nyairo, M. E., & Omwando, M. N. (2026). Development of an Easy-To-Use Cybersecurity Framework for Safe and Effective IoT Adoption Among

SMEs in Nairobi County, Kenya. *International Journal of Social Sciences Management and Entrepreneurship (IJSSME)*, 10(1).

12. Palo Alto Networks, n.d. Palo Alto Networks IoT security: Visibility and policy enforcement for connected devices. [online] Available at: <https://www.paloaltonetworks.com/network-security/what-is-iot-security> [Accessed 25 February 2026].
13. Parra, D. T., & Guerrero, C. D. (2020, June). Decision-making IoT adoption in SMEs from a technological perspective. In *2020 15th Iberian Conference on Information Systems and Technologies (CISTI)* (pp. 1-6). IEEE.
14. Suci, A. D., Tudor, A. I. M., Chişu, I. B., Doleac, L., & Brătucu, G. (2021). IoT technologies as instruments for SMEs' innovation and sustainable growth. *Sustainability*, 13(11), 6357.
15. University of Nairobi, 2024. The "Big 5" Strategic Initiatives. [online] Available at: <https://www.uonbi.ac.ke/news/university-nairobi-announces-%E2%80%9Cbig-5%E2%80%9D-major-new-projects> [Accessed 20 February 2026].

APPENDICES

Appendix A: SentiNet System User Guide

This guide provides step-by-step instructions for downloading, installing, and operating the SentiNet IoT Security Monitoring System. The guide is intended for IT administrators and network managers within SME environments.

A.1 System Requirements

Before installation, ensure the following prerequisites are met on the development or deployment machine:

Table 3: System Requirements

Requirement	Specification
Operating System	Ubuntu 20.04 or later (Linux recommended); Windows 10+ with WSL2
RAM	Minimum 8 GB (16 GB recommended for Docker simulation)
Storage	At least 10 GB free disk space
Processor	Intel Core i5 or equivalent

A.2 Accessing the System

Open a modern web browser and navigate to <http://127.0.0.1:5000>. The SentiNet landing page is displayed, presenting the system name and a brief description of its purpose.

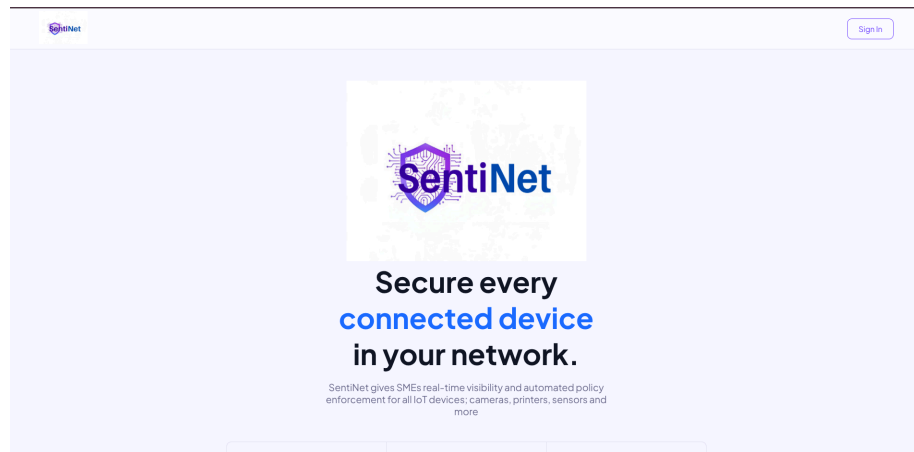


Figure A.2: SentiNet landing page

A.3 Creating a New Account

New users must create an administrator account before accessing the dashboard. To register:

1. Click Get Started on the landing page or navigate to <http://127.0.0.1:5000/signin>.
2. Select the Create Account tab on the authentication page.
3. Enter the full name, organisation name, and select the user role from the dropdown.
4. Enter a valid email address and a strong password, then confirm the password.
5. Tick the terms of agreement checkbox to confirm authorised use of the scanning features.
6. Click Create Account to complete registration.

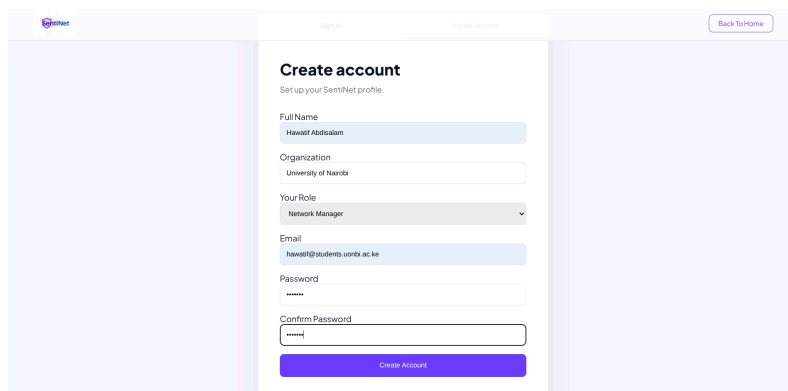
The image shows a web form titled "Create account" with the subtitle "Set up your SentiNet profile." The form is divided into two main sections: "Sign In" and "Create Account", with the "Create Account" tab selected. The form fields include: "Full Name" (text input with "Hassan Abdalamin" entered), "Organization" (text input with "University of Nairobi" entered), "Your Role" (dropdown menu with "Network Manager" selected), "Email" (text input with "hassan@students.uonbi.ac.ke" entered), "Password" (password input field), and "Confirm Password" (password input field). A "Create Account" button is at the bottom. A "Back To Home" link is in the top right corner. The SentiNet logo is in the top left corner.

Figure A.3: Account registration form

A.4 Signing In

Registered users sign in using their email address and password via the Sign In tab on the authentication page. On successful authentication, the system redirects to the main dashboard. The administrator name and role are displayed in the sidebar footer.

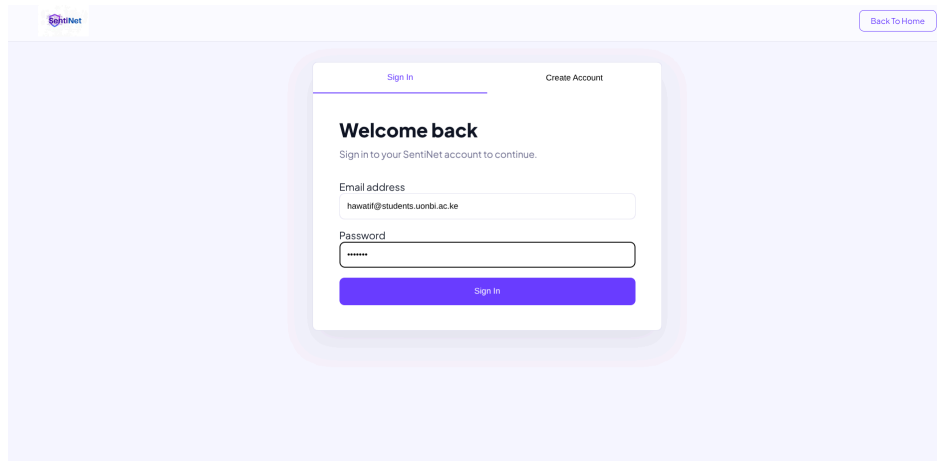


Figure A.4: Sign in page

A.5 Dashboard Overview

The dashboard is the main workspace for scanning networks and reviewing device security status. The interface consists of a left sidebar for navigation and a main content area containing the Network Scanner input, summary statistics, filter controls, and device cards.

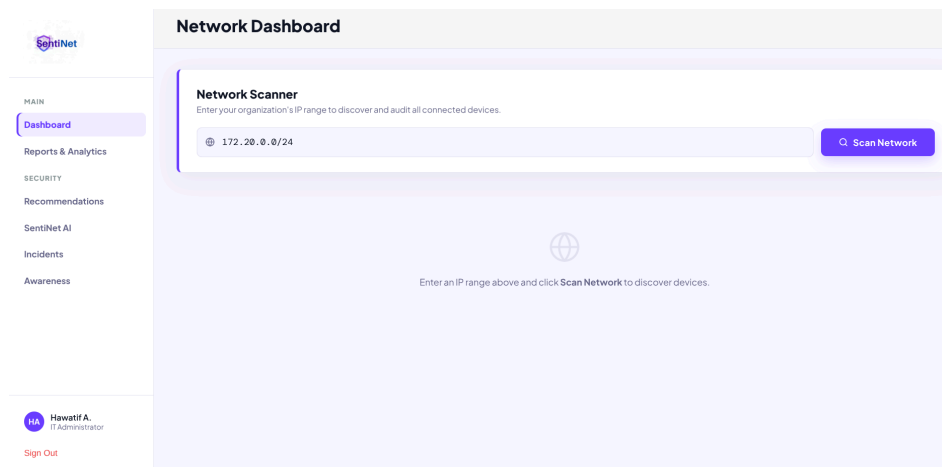


Figure A.5: Main dashboard before scanning

The summary statistics panel at the top of the dashboard displays the following metrics after a scan is completed:

Table 4: Summary statistics

Statistic	Meaning
Total Devices	The total number of devices discovered in the scanned network range.
IoT Devices	The number of devices classified as IoT, including cameras, printers, and wireless access points.
High Risk	The number of devices assigned a high or critical risk level, requiring immediate attention.
NVD Findings	The total number of CVE vulnerabilities retrieved from the National Vulnerability Database for all scanned devices.

A.6 Running a Network Scan

The Network Scanner accepts an IP range in CIDR notation. To scan the Docker simulation environment, enter 172.20.0.0/24 in the scan input field and click Scan Network. For a live office network, enter the appropriate subnet range, for example 192.168.1.0/24.

A progress indicator is displayed while the scan is running. The scan performs the following operations in sequence: Nmap network discovery, ML classification, credential and firmware checks, NVD vulnerability lookups, and risk scoring. The process typically takes between 30 and 90 seconds depending on the number of devices and NVD response times.

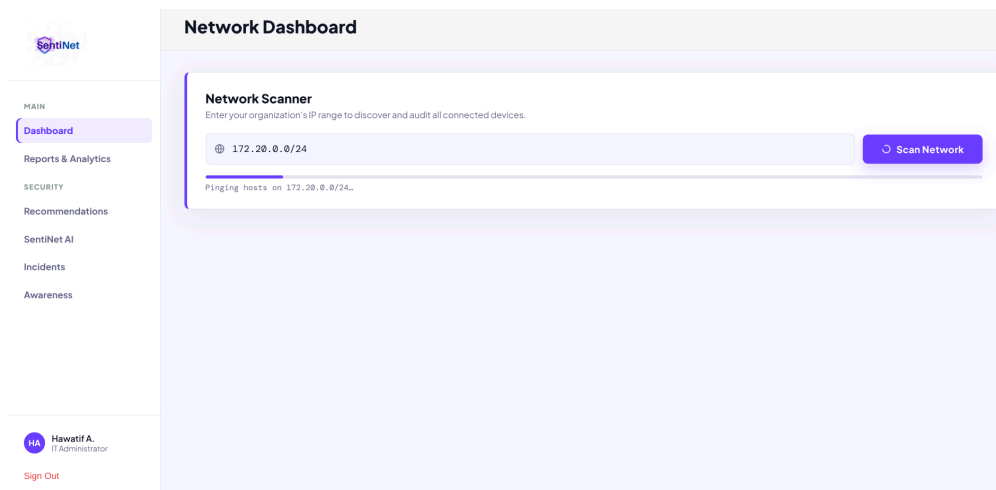


Figure A.6: Network scan in progress

Once the scan completes, the summary statistics update and device cards populate the dashboard below the scanner panel.

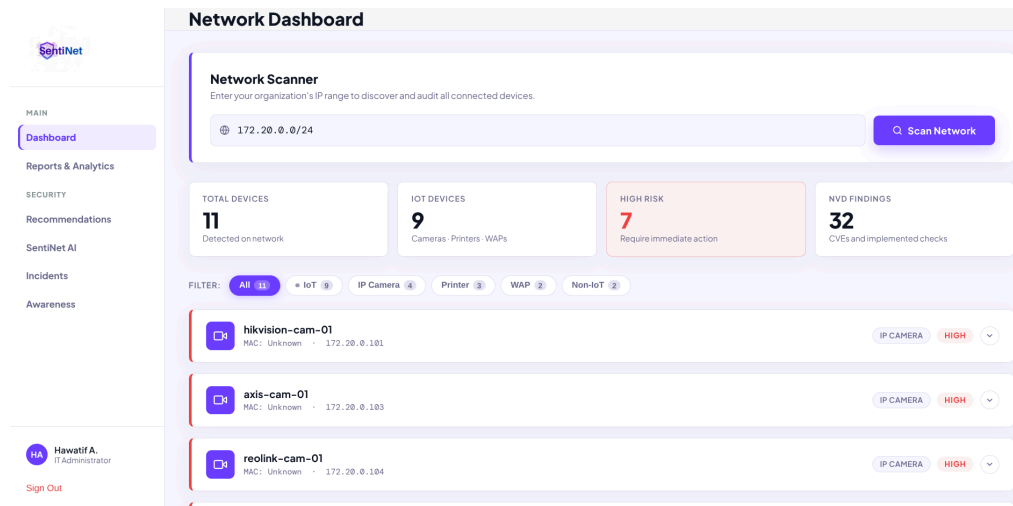


Figure A.7: Dashboard showing scan results with device cards

A.7 Device Cards and Security Findings

Each discovered device is presented as an expandable card displaying the following information:

1. Device name, IP address, and MAC address
2. Device type (IP Camera, Printer, Wireless AP, Computer) and manufacturer
3. Risk level indicator (Low, Medium, High) and ML classification confidence score
4. Open ports with service names and safety indicators
5. Security findings including local policy checks and NVD CVE findings with CVSS scores and severity levels

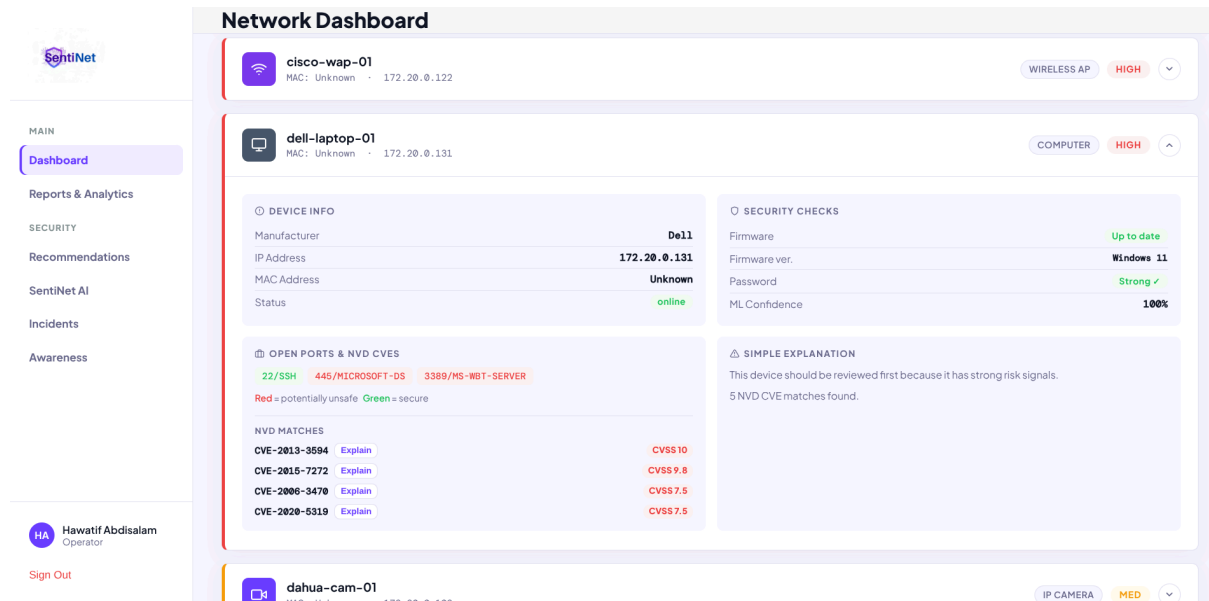


Figure A.8: Expanded device card showing security findings and CVE data

Security findings are grouped into two categories: Local Checks (credential status, firmware version, risky ports) and NVD Findings (CVE identifiers, CVSS scores, and severity classifications retrieved from the National Vulnerability Database).

A.8 Filtering Devices

Filter chips above the device cards allow administrators to narrow the view by device category. Available filters are: All, IoT, IP Camera, Printer, WAP, and Non-IoT. Clicking a filter chip updates the device list immediately without requiring a new scan.

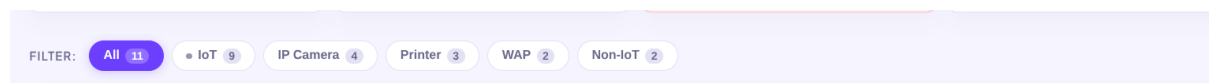


Figure A.9: Device list filtered by IP Camera category

A.9 Reports and Analytics

The Reports and Analytics page is accessible from the sidebar navigation. It presents visual summaries of scan results through interactive charts and a scan history table. The following components are displayed:

1. Risk Distribution: A chart categorizing all scanned devices by risk level (Low, Medium, High).
2. Device Mix: A chart showing the breakdown of discovered device types across the network.
3. Scan History: A table of previous scan results including the network range, total devices, IoT device count, and high-risk device count for each recorded scan.
4. Download PDF Report: A button to generate and download a full sentinet-report.pdf containing all device details, port information, and CVE findings.

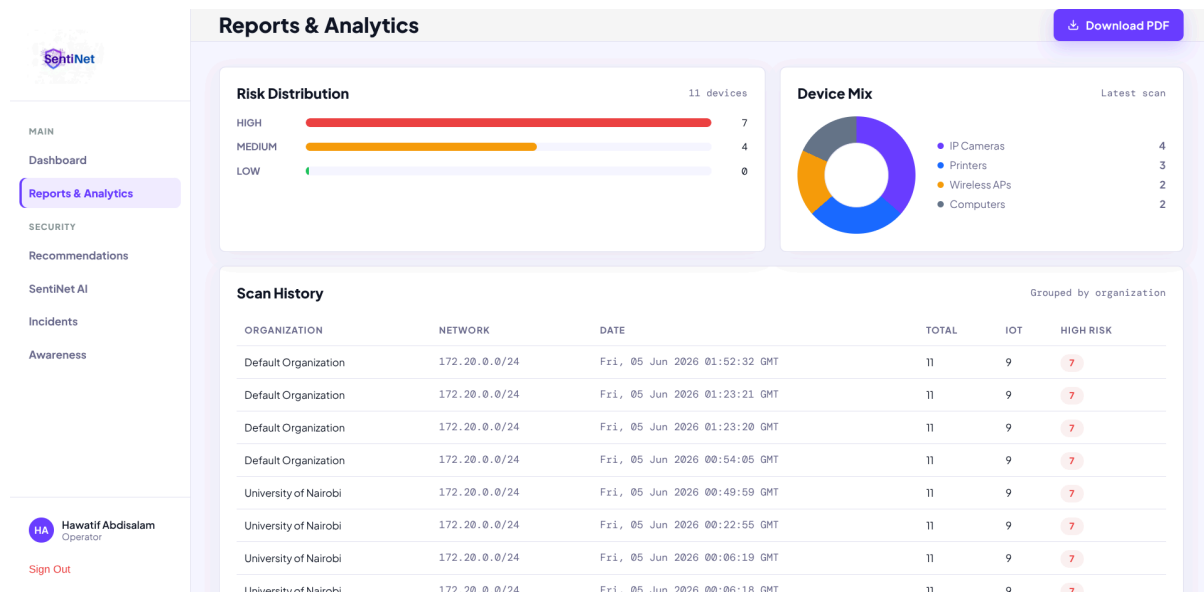


Figure A.10: Reports and Analytics page with risk distribution and device mix charts

A.10 Security Centre

The Security Centre is accessible from the sidebar and provides three additional modules for administrators:

1. Recommendations: A prioritized list of security actions derived from the scan results, including explanations of each issue and step-by-step remediation guidance.
2. Incidents: A tracking view listing high and medium severity issues as open incidents, each with an assigned remediation action.
3. Security Awareness: A set of short educational modules covering IoT security topics including default passwords, firmware patching, Telnet exposure, and network segmentation.

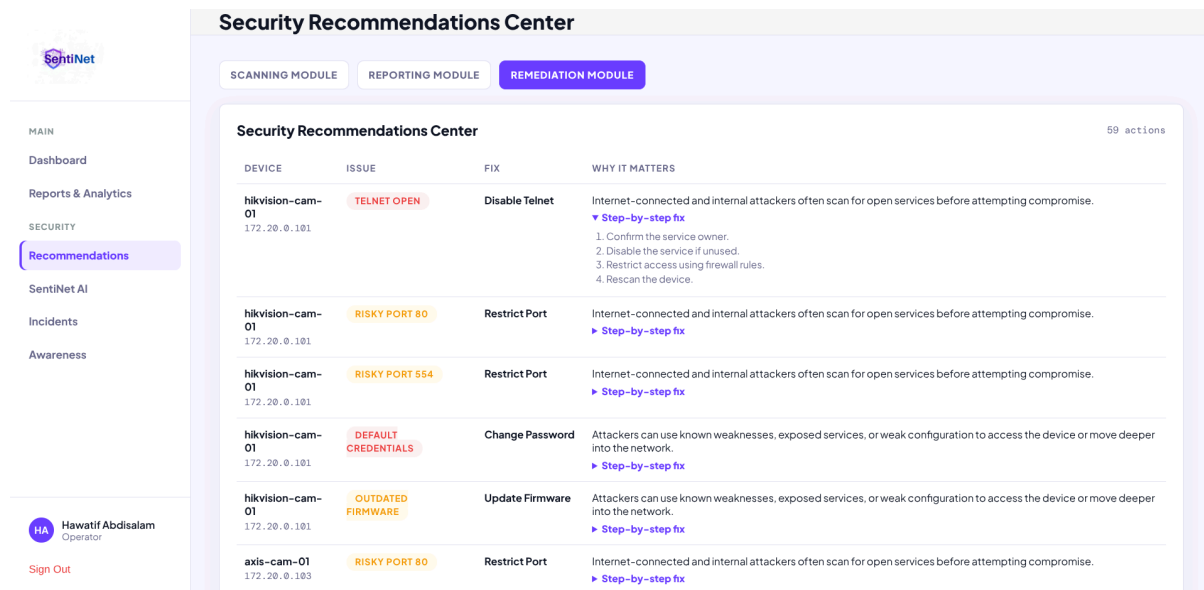


Figure A.11: Security Centre showing prioritized recommendations

A.11 SentiNet AI Assistant

The SentiNet AI Assistant is accessible from the Security Centre. It accepts natural language questions about the latest scan results and provides contextual security guidance. Example questions include:

1. "Which device is most vulnerable?"
2. "What CVEs were found?"
3. "How do I fix the default credential issue?"
4. "What is the compliance status of the network?"

The assistant operates in two modes: if an OpenAI API key is configured in the .env file, it uses the OpenAI model for enhanced natural language responses; otherwise it falls back to a local rule-based engine that answers based on the scan data.

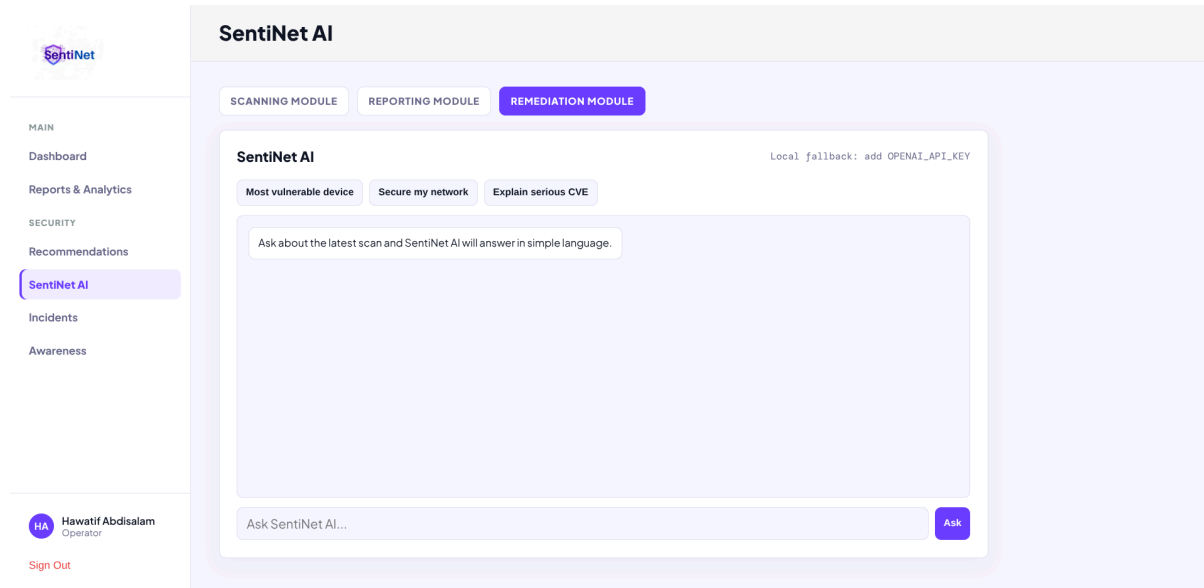


Figure A.12: SentiNet AI assistant responding to a security query

A.12 Incidents

The Incidents view lists all high and medium severity security issues detected during the last scan as open incidents. Each incident displays the affected device, the issue type, and the assigned remediation action.

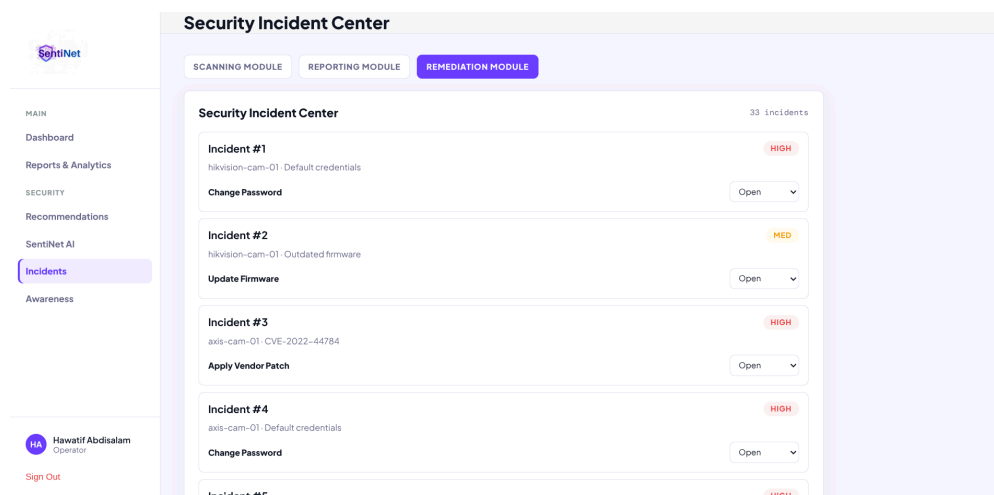


Figure A.13: Incidents view listing open security issues

A.12.2 Security Awareness

1. The Security Awareness module provides short educational content on key IoT security topics. The available modules are:
2. Default Passwords: Covers the risks of unchanged default credentials and how to update them on common IoT devices.
3. Firmware Patching: Explains the importance of keeping device firmware up to date and how to check for available updates.
4. Telnet Exposure: Describes the security risks of open Telnet ports and how to disable Telnet in favour of more secure protocols.
5. Network Segmentation: Introduces the concept of isolating IoT devices on a separate network segment to limit the impact of a compromised device.

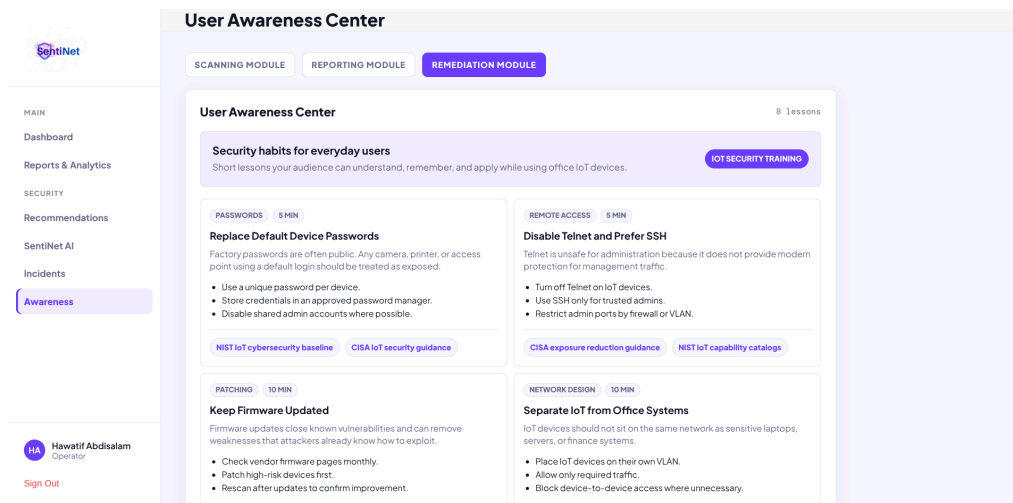


Figure A.14: Security Awareness module showing available IoT security topics

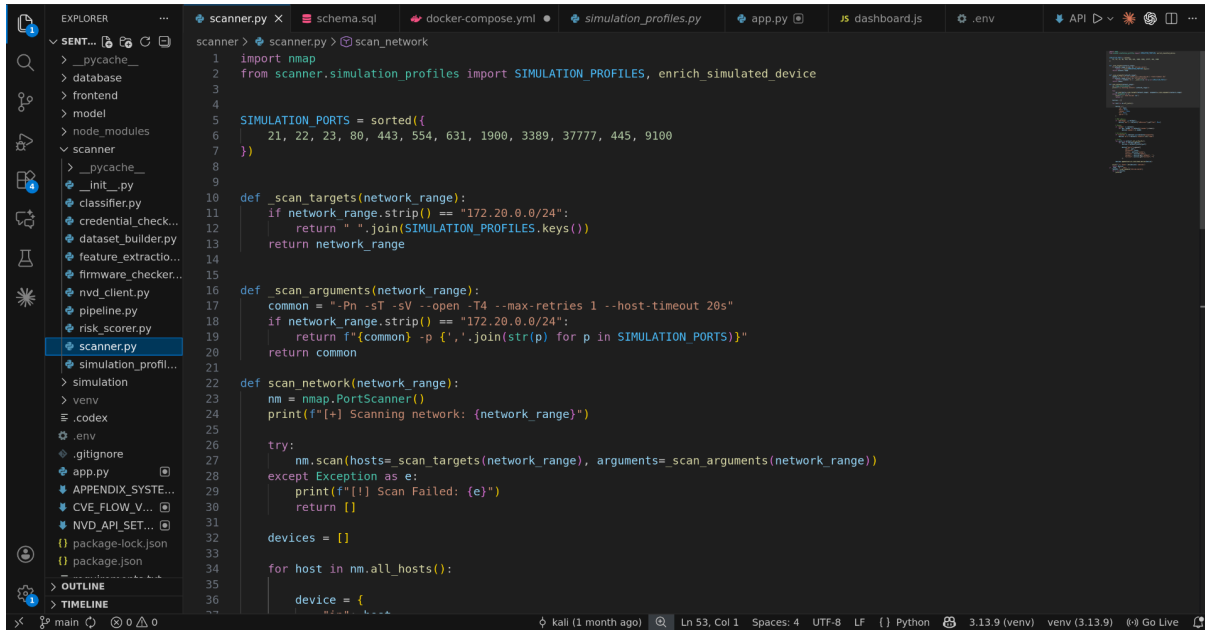
A.13 Troubleshooting

Table 5: Troubleshooting

Problem	Possible Cause	Solution
Dashboard does not load	Flask server not running	Activate the virtual environment and run python3 app.py from the project root.

Database connection error on startup	MySQL / XAMPP not running	Start XAMPP, ensure MySQL is active, and confirm the sentinel database exists in phpMyAdmin.
Scan returns no devices	Docker containers not running	Run docker compose up -d from the simulation/ directory and wait for all containers to reach Up status.
Scan is very slow or times out	Missing NVD API key or network latency	Add the NVD API key to the .env file. Ensure the host machine has internet access for NVD queries.
Permission denied during Nmap scan	Insufficient OS permissions for raw socket scans	Run the application with sudo or ensure Nmap is configured for TCP connect scans, which do not require elevated privileges.
ML model file not found	Classifier has not been trained	Run python3 scanner/classifier.py to generate the model files in the model/ directory.
CVE data missing for devices	NVD rate limiting or API key absent	Add NVD_API_KEY to .env or wait before rescanning. The free tier allows 5 requests per 30 seconds without a key.

Appendix B: Sample Code



```
1 import nmap
2 from scanner.simulation_profiles import SIMULATION_PROFILES, enrich_simulated_device
3
4
5 SIMULATION_PORTS = sorted([
6     21, 22, 23, 80, 443, 554, 631, 1900, 3389, 37777, 445, 9100
7 ])
8
9
10 def _scan_targets(network_range):
11     if network_range.strip() == "172.20.0.0/24":
12         return " ".join(SIMULATION_PROFILES.keys())
13     return network_range
14
15
16 def _scan_arguments(network_range):
17     common = "-Pn -sT -sV --open -T4 --max-retries 1 --host-timeout 20s"
18     if network_range.strip() == "172.20.0.0/24":
19         return f"{common} -p {' '.join(str(p) for p in SIMULATION_PORTS)}"
20     return common
21
22
23 def scan_network(network_range):
24     nm = nmap.PortScanner()
25     print(f"[+] Scanning network: {network_range}")
26
27     try:
28         nm.scan(hosts=_scan_targets(network_range), arguments=_scan_arguments(network_range))
29     except Exception as e:
30         print(f"[!] Scan Failed: {e}")
31         return []
32
33     devices = []
34
35     for host in nm.all_hosts():
36         device = {
37             "ip": host,
```

Figure B.1 Sample code